

Experiment no. 4

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

Blockchain is a decentralized and distributed ledger technology that records transactions in an immutable and transparent manner. Ethereum extends blockchain functionality by allowing developers to create smart contracts, which are self-executing programs deployed on the blockchain. Solidity is a high-level, contract-oriented programming language used to write these smart contracts.

This experiment focuses on understanding the fundamental concepts of Solidity and implementing them through hands-on programming to design secure and efficient smart contracts.

1. Primitive Data Types and Variables in Solidity

Solidity provides several primitive data types that serve as the building blocks for smart contract development:

- **Integer Types (uint, int)**
Used to store numeric values. uint stores unsigned integers (non-negative), while int stores signed integers. Commonly used sizes include uint256, which is optimized for Ethereum Virtual Machine (EVM).
- **Boolean (bool)**
Stores logical values true or false, typically used in conditions and decision-making.
- **Address (address)**
Represents a 20-byte Ethereum account or contract address. It is commonly used for identifying users, transferring Ether, and enforcing ownership.
- **Bytes and String (bytes, string)**
bytes is used for raw binary data, while string is used for storing textual information.

Variables in Solidity are classified as:

- **State Variables:** Stored permanently on the blockchain.
- **Local Variables:** Declared inside functions and exist temporarily.
- **Global Variables:** Predefined variables like msg.sender, msg.value, and block.timestamp that provide blockchain-related information.

2. Functions and Function Types

Functions define the logic of a smart contract. They can accept inputs, perform computations, and return outputs.

Special function types include:

- pure functions: Do not read or modify state variables. They only use input parameters and internal logic.
- view functions: Can read state variables but cannot modify them.

Using pure and view functions helps reduce gas costs and improves contract efficiency.

3. Inputs and Outputs to Functions

Solidity functions can take one or more input parameters and return one or more output values. Inputs allow data to be passed into the contract, while outputs return computed results to the caller.

Functions can also use named return variables, which enhance code readability and simplify debugging. This feature is especially useful when returning complex data structures.

4. Function Visibility, Modifiers, and Constructors

Function Visibility

Solidity provides access control through visibility specifiers:

- public: Can be accessed both internally and externally.
- private: Accessible only within the same contract.
- internal: Accessible within the contract and derived contracts.
- external: Can only be called from outside the contract.

Modifiers

Modifiers are reusable pieces of code used to alter function behavior. They are commonly used to enforce conditions such as ownership or authorization (e.g., `onlyOwner`).

Constructors

A constructor is a special function executed only once during contract deployment. It is typically used to initialize state variables, such as assigning the deployer as the contract owner.

5. Control Flow Statements

Solidity supports standard control flow mechanisms:

- Conditional Statements (if-else)
Used to execute code based on conditions, such as validating balances before transactions.
- Loops (for, while, do-while)
Used to repeat execution of code blocks. However, excessive looping increases gas consumption, so loops must be used carefully in smart contracts.

6. Data Structures in Solidity

Solidity provides several data structures for efficient data management:

- Arrays
Store ordered collections of elements. Arrays can be fixed-size or dynamic.
- Mappings
Store key-value pairs for fast lookups, such as `mapping(address => uint)` for account balances. Mappings do not support iteration.
- Structs
Group related variables into a single custom data type, improving code organization.
- Enums
Define a set of named constants, enhancing code readability and reducing logical errors.

7. Data Locations

Solidity uses three data locations to manage variable storage:

- storage: Permanent data stored on the blockchain (state variables).
- memory: Temporary data used during function execution.
- calldata: Read-only and gas-efficient location used for external function parameters.

Choosing the correct data location is crucial for optimizing gas usage and contract performance.

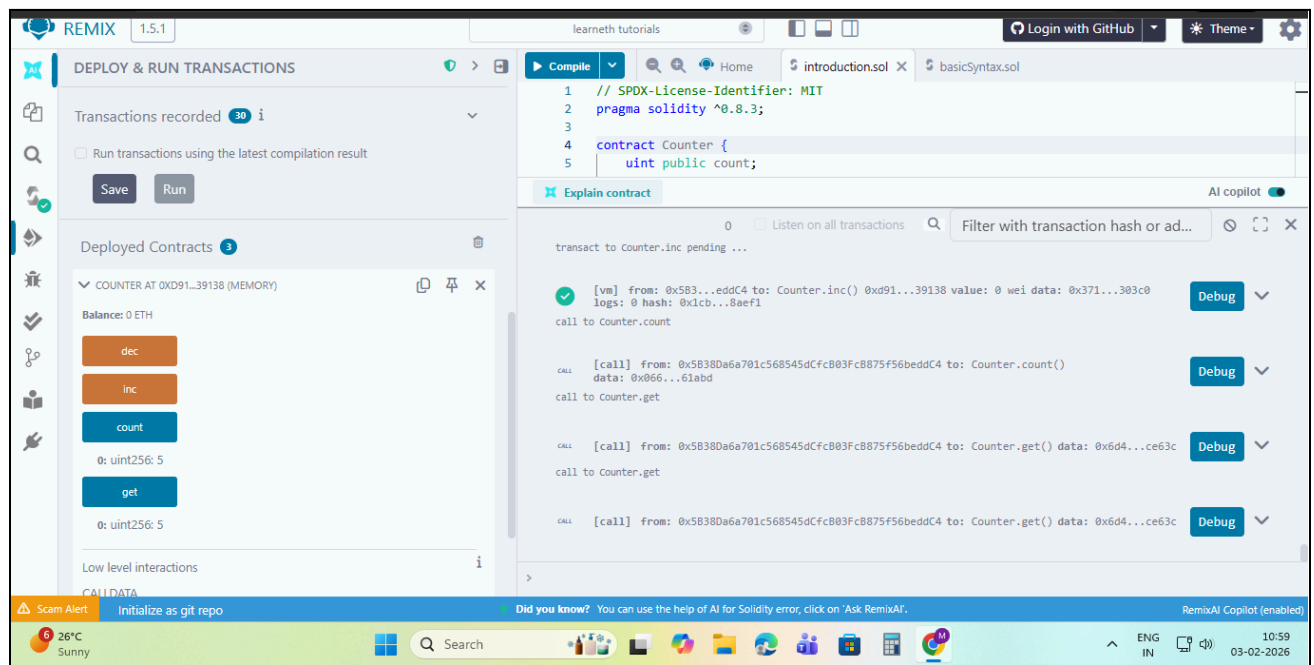
8. Transactions, Ether, and Gas

- Ether and Wei
Ether is the native cryptocurrency of Ethereum. All values are internally handled in Wei, where $1 \text{ Ether} = 10^{18} \text{ Wei}$, allowing precise financial calculations.
- Gas and Gas Price
Gas measures the computational effort required to execute transactions or smart contracts. The gas price determines how much Ether is paid per unit of gas.
- Sending Transactions
Transactions are used to transfer Ether or interact with smart contracts. Methods like `transfer()`, `send()`, and `call()` are used, with `call()` offering greater flexibility and control.

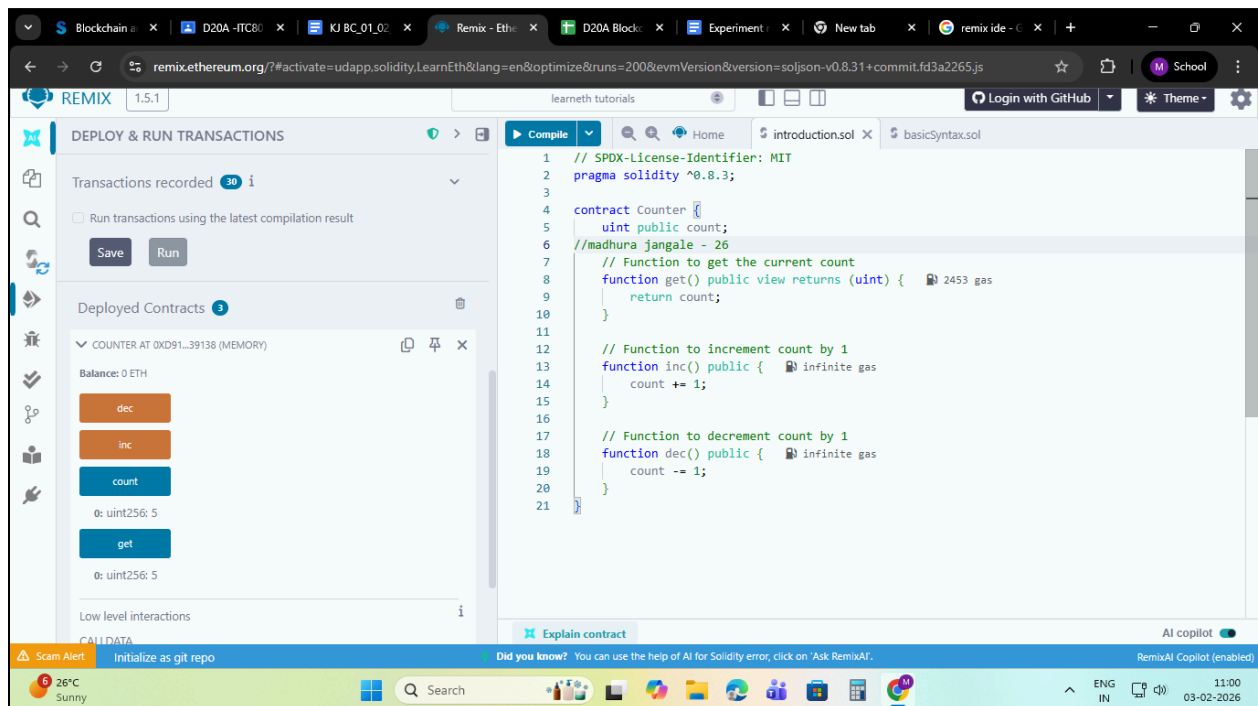
Efficient contract design minimizes gas consumption and ensures secure execution.

Implementation:

1. basic.sol



2. introduction.sol



The screenshot shows the REMIX IDE interface. On the left, the 'LEARNETH' sidebar displays the '2. Basic Syntax' assignment (2 / 19). The assignment instructions are:

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Below the instructions are buttons for 'Check Answer', 'Show answer', and 'Next'. A green message at the bottom states 'Well done! No errors.'.

The main editor shows the Solidity code for 'basicSyntax.sol':`1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and
3 pragma solidity ^0.8.3;
4 //Madhura Jangale - 26
5
6
7
8
9 contract MyContract{
10 | string public name = "Alice";
11 }`

3. PrimitiveDataTypes.sol

The screenshot shows the REMIX IDE interface. On the left, the 'LEARNETH' sidebar displays the '3. Primitive Data Types' assignment (3 / 19). The assignment instructions are:

1. that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

A tip is provided: 'Tip: Look at the other address in the contract or search the internet for an Ethereum address.' Below the instructions are buttons for 'Check Answer', 'Show answer', and 'Next'. A green message at the bottom states 'Well done! No errors.'.

The main editor shows the Solidity code for 'primitiveDataTypes.sol':`2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5 bool public boo = true;
6 //Madhura Jangale - 26
7
8 uint8 public u8 = 1;
9 uint public u256 = 456;
10 uint public u = 123; // uint is an alias for uint256
11
12 int8 public i8 = -1;
13 int public i256 = 456;
14 int public i = -123; // int is same as int256
15
16 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
17 address public newAddr = 0x00;
18
19 // Default values
20 // Unassigned variables have a default value
21 bool public defaultBoo; // false
22 uint public defaultUInt; // 0
23 int public defaultInt; // 0
24 uint8 public newU = 0;
25 int public neg = -26;
26 address public defaultAddr; // 0x00
27 }`

4. Variables.sol

REMIX1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

★ Assignment

1. Create a new public state variable called `blockNumber`.

2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compile

basicSyntax.sol

primitiveDataTypes.sol

variables.sol 3

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Madhura Jangale";
7     uint public num = 26;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

5. readandWrite.sol

REMIX1.5.1

learneth tutorials

Login with

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.

2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

basicSyntax.sol

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8     // You need to send a transaction to write to a state variable.
9     function set(uint _num) public { 22536 gas
10         num = _num;
11     }
12
13     // You can read from a state variable without sending a transaction.
14     function get() public view returns (uint) { 2475 gas
15         return num;
16     }
17
18     function get_b() public view returns (bool) { 2539 gas
19         return b;
20     }
21 }
```

6. ViewAndPure.sol

REMIX 1.5.1

learneth tutorials

Login

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

asicSyntax.sol

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //madhura jangale - 26
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addToX2(uint y) public {
18        x=y+x;
19    }
20 }

```

7. ModifiersandConstructors.sol

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compile

primitiveDataTypes.sol

variables.sol 3

readAndWrite.sol

```

26 // modifiers can take inputs. this modifier checks that the
27 // address passed in is not the zero address.
28 modifier validAddress(address _addr) {
29     require(_addr != address(0), "Not valid address");
30     _;
31 }
32
33 function changeOwner(address _newOwner) public onlyOwner validAddress {
34     owner = _newOwner;
35 }
36 //madhura jangale - 26
37 function increaseX(uint _value) public onlyOwner {
38     x += _value;
39 }
40
41 // Modifiers can be called before and / or after a function.
42 // This modifier prevents a function from being called while
43 // it is still executing.
44 modifier noReentrancy() {
45     require(!locked, "No reentrancy");
46
47     locked = true;
48     _;
49     locked = false;
50 }
51
52 function decrement(uint i) public noReentrancy {
53     //

```

8. inputsandOutputs.sol

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs

8 / 19

size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

★ **Assignment**

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

readAndWrite.sol

viewAndPure.sol

modifiersAndConstructors.sol

```

58     )
59     {
60         (uint i, bool b, uint j) = returnMany();
61
62         // Values can be left out.
63         (uint x, , uint y) = (4, 5, 6);
64
65         return (i, b, j, x, y);
66     }
67
68     // Cannot use map for neither input nor output
69
70     // Can use array for input
71     function arrayInput(uint[] memory _arr) public {}  infinite gas
72
73     // Can use array for output
74     uint[] public arr;
75
76     function arrayOutput() public view returns (uint[] memory) {  infinite gas
77         return arr;
78     }
79     //madhura jangale - 26
80     function returnTwo() public pure returns (int m, bool mm) {  472 gas
81         m = -2;
82         mm = true;
83     }
84 }

```

9. Visibility.sol

LEARNETH

Tutorials list

Syllabus

6. Visibility

9 / 19

you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

★ **Assignment**

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile

ifElse_answer.sol

visibility.sol

```

48     string private privateVar = "my private variable";
49     string internal internalVar = "my internal variable";
50     string public publicVar = "my public variable";
51     // State variables cannot be external so this code won't compile.
52     // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     // function testPrivateFunc() public pure returns (string memory) {
59     //     return privateFunc();
60     // }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66     //Madhura Jangale - 26
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }

```

10.IfElse.sol

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compile

viewAndPure.sol

modifiersAndConstructors.sol

```

8      } else if (x < 20) {
9          return 1;
10     } else {
11         return 2;
12     }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24 //madhura jangale - 2d
25 function evenCheck(int _x) public pure returns (bool) {
26     if(_x % 2 == 0) {
27         return true;
28     }
29     else{
30         return false;
31     }
32 }
33

```

11. Loops.sol

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

Watch a video tutorial on Loop statements.

★ Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

ure.sol

modifiersAndConstructors.sol

input

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Loop {
5      uint public count;
6      //madhura jangale - 2d
7      function loop() public {
8          // for loop
9          for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20
21         // while loop
22         uint j;
23         while (j < 10) {
24             j++;
25         }
26     }
27 }

```

12. Arrays.sol

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

visibility.sol ifElse.sol loops.sol arrays.sol

```

7   uint[] public arr2 = [1, 2, 3];
8   // Fixed sized array, all elements initialize to 0
9   uint[10] public myFixedSizeArr;
10  uint[3] public arr3 = [0,1,2];
11  function get(uint i) public view returns (uint) {
12      return arr[i];
13  }
14
15  // Solidity can return the entire array.
16  // But this function should be avoided for
17  // arrays that can grow indefinitely in length.
18  function getArr() public view returns (uint[3] memory) {
19      return arr3;
20  }
21
22  function push(uint i) public {
23      // Append to array
24      // This will increase the array length by 1.
25      arr.push(i);
26  }
27
28  function pop() public {
29      // Remove last element from array
30      // This will decrease the array length by 1
31      arr.pop();
32  }
33

```

13. Mappings.sol

REMUX 1.5.1

learneth tutorials

Login with GitHub

LEARNETH

Tutorials list

Syllabus

8.2 Data Structures - Mappings

13 / 19

arrays section.

[Watch a video tutorial on Mappings.](#)

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Compile

visibility.sol ifElse.sol loops.sol arrays.sol

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Mapping {
5      // Mapping from address to uint
6      mapping(address => uint) public balances;
7      //Madhura Jangale - 26
8      function get(address _addr) public view returns (uint) {
9          // Mapping always returns a value.
10         // If the value was never set, it will return the default value.
11         return balances[_addr];
12     }
13
14     function set(address _addr) public {
15         // Update the value at this address
16         balances[_addr] = _addr.balance;
17     }
18
19     function remove(address _addr) public {
20         // Reset the value to the default value.
21         delete balances[_addr];
22     }
23 }
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;

```

14.Struct.sol

REMUX 1.5.1

learneth tutorials

Login with GitHub

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

14 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile

mappings.sol

structs.sol

structs_answer.sol

```

27
28
29
30
31 // Solidity automatically created a getter for 'todos' so
32 // you don't actually need this function.
33 function get(uint _index) public view returns (string memory text, bool completed)
34 {
35     Todo storage todo = todos[_index];
36     return (todo.text, todo.completed);
37 }
38
39 // update text
40 function update(uint _index, string memory _text) public {
41     infinite gas
42     Todo storage todo = todos[_index];
43     todo.text = _text;
44 }
45
46 // update completed
47 function toggleCompleted(uint _index) public {
48     28995 gas
49     Todo storage todo = todos[_index];
50     todo.completed = !todo.completed;
51 }
52
53 //Madhura - 26
54 function remove(uint _index) public {
55     infinite gas
56     delete todos[_index];
57 }
58
59
60

```

15.enums.sol

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

- Define an enum type called `Size` with the members `S`, `M`, and `L`.
- Initialize the variable `sizes` of the enum type `Size`.
- Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

structs.sol

structs_answer.sol

e

```

30
31 function get() public view returns (Status) {
32     return status;
33 }
34
35 function getSize() public view returns (Size) {
36     return sizes;
37 }
38
39 //Madhura Jangale - 26
40
41 // Update status by passing uint into input
42 function set(Status _status) public {
43     undefined
44     status = _status;
45 }
46
47 // You can update to a specific enum like this
48 function cancel() public {
49     24494 gas
50     status = Status.Canceled;
51 }
52
53 // delete resets the enum to its first value, 0
54 function reset() public {
55     24383 gas
56     delete status;
57 }
58
59
60

```

16. dataLocations.sol

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.

3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.

4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

dataLocations.sol 2 X

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) public myStructs;
11    //Madhura Jangale - 26
12    function f() public returns (MyStruct memory, MyStru
13        // call _f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myMemStruct = MyStruct(0);
20        MyStruct memory myMemStruct2 = myMemStruct;
21        myMemStruct2.foo = 1;
22
23        MyStruct memory myMemStruct3 = myStruct;
24        myMemStruct3.foo = 3;
25        return (myStruct, myMemStruct2, myMemStruct3);
26    }
27

```

17.etherandWei.sol

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.

2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compile

etherAndWei.sol X

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 //Madhura Jangale - 26
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }

```

18.gasandGasPrice.sol

LEARNETH

Tutorials list

Syllabus

10.2 Transactions - Gas and Gas Price

18 / 19

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas profiler` to check for the gas cost of transactions.

Check Answer

Show answer

Next

Well done! No errors.

Compile

etherAndWeisol

gasAndGasPrice.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7     //Madhura Jangale - 26
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {  infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }

```

19. SendingEther.sol

LEARNETH

Tutorials list

Syllabus

10.3 Transactions - Sending Ether

19 / 19

public and payable without any parameters or function code.

4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer

Show answer

Next

Well done! No errors.

Compile

sendingEther.sol 4

```

43 }
44
45 function sendViaCall(address payable _to) public payable {  undefined
46     // Call returns a boolean value indicating success or failure.
47     // This is the current recommended method to use.
48     (bool sent, bytes memory data) = _to.call{value: msg.value}("");
49     require(sent, "Failed to send Ether");
50 }
51
52 //Madhura Jangale - 26
53 contract Charity {
54     address public owner;
55
56     constructor() { 
57         owner = msg.sender;
58     }
59
60     function donate() public payable {} 
61
62     function withdraw() public { 
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call{value: amount}("");
66         require(sent, "Failed to send Ether");
67     }
68 }

```

Conclusion:

This experiment provides hands-on experience in Solidity programming by exploring core blockchain concepts, data types, control structures, and transaction handling. Understanding these fundamentals enables the design and deployment of secure, efficient, and reliable smart contracts on the Ethereum blockchain.